

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE CODE: CSA14

COURSE NAME: Compiler Design

COURSE OUTCOME COVERED

CO4: *Examine intermediate code generation techniques to enhance code optimization (BL4)*

Assessment Tool Weightage: 10%

QUESTIONS:

Total Marks:50

Annexure A

DATA INTERPRETATION

Code of Conduct:

I Jerold Samuel J certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Jerold Samuel J

Annexure A

DATA INTERPRETATION

Q1. Intermediate Code Analysis and Optimization Given the three-address code:

$t1 = x + y$ $t2$

$= t1 * z$ $t3 =$

$t2 / w$ $t4 =$

$t3 - p$

Analyze the

sequence of
operations
and
construct
the
equivalent
expression
tree.

Evaluate
how
temporary
variables
improve
intermediate
code
generation
and reduce
repeated
computations.

ANSWER:

Intermediate Code Analysis and Optimization

Given three-address code:

```
t1 = x + y
t2 = t1 * z
t3 = t2 / w
t4 = t3 - p
```

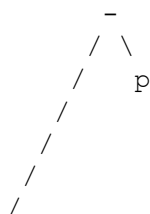
Sequence of Operations

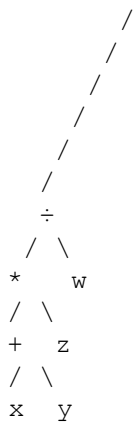
1. $t1 = x + y \rightarrow$ adds x and y .
2. $t2 = t1 * z \rightarrow$ multiplies the result by z .
3. $t3 = t2 / w \rightarrow$ divides the result by w .
4. $t4 = t3 - p \rightarrow$ subtracts p .

Equivalent Expression

$((x + y) * z) / w - p$

Expression Tree





Role of Temporary Variables

The temporary variables t_1 , t_2 , t_3 , and t_4 store intermediate results. They:

- Break a complex expression into simple operations.
- Make intermediate code easier for the compiler to process.
- Allow intermediate results to be reused.
- Help reduce repeated computations.
- Make optimization and register allocation easier.

Conclusion: Temporary variables provide a structured representation of complex expressions and help the compiler generate efficient intermediate code.

Q2. Symbol Table and Memory Allocation

Given the following symbol table entries:

Variable	Data Type	Size (Bytes)
p	char	1
q	double	8
r	int	4
s	float	4

Interpret how the compiler allocates memory for these variables and calculate the total memory required. Analyze how alignment and data types affect memory organization.

ANSWER:

Symbol Table and Memory Allocation

Given:

Variable	Data Type	Size
p	char	1 byte

q double 8 bytes

r int 4 bytes

s float 4 bytes

Memory Calculation

p = 1 byte

q = 8 bytes

r = 4 bytes

s = 4 bytes

Total = 1 + 8 + 4 + 4

= 17 bytes

Therefore, the **total memory required is 17 bytes**, before considering padding and alignment.

Memory Allocation

The compiler uses the symbol table to determine:

- Variable name
- Data type
- Storage size
- Memory location/alignment requirements

The char variable requires only 1 byte, whereas double requires 8 bytes.

Effect of Alignment

Alignment means placing data at memory addresses suitable for its data type. For example, a double may require an 8-byte-aligned address on many systems.

Because of alignment, the compiler may insert **padding bytes** between variables. Therefore, the actual memory occupied by the variables can sometimes be greater than the simple total of 17 bytes.

Conclusion: Data types determine the basic storage requirement, while alignment can introduce additional padding to organize memory efficiently.

Q3. Optimization in Intermediate Code

Given the intermediate code:

$t1 = m * n$

$t2 = p + q$

$t3 = m * n$

$t4 = t2 - t3$

Trace the execution step-by-step and identify optimization opportunities such as common subexpression elimination and redundant computation removal. Explain how optimization improves execution efficiency.

ANSWER:

Optimization in Intermediate Code

Given:

$t1 = m * n$

$t2 = p + q$

$t3 = m * n$

$t4 = t2 - t3$

Step-by-Step Execution

Step 1: $t1$

$= m * n$

The values of m and n are multiplied and stored in $t1$.

Step 2:

$t2 = p + q$

The values of p and q are added and stored in $t2$.

Step 3: $t3$

$= m * n$

The multiplication $m * n$ is performed again.

Step 4: $t4$

$= t2 - t3$

The value of $t3$ is subtracted from $t2$.

Optimization Opportunity

The expression:

$m * n$ occurs

twice: $t1 = m$

$* n$

...

$t3 = m * n$

If m and n have not changed, the second calculation is unnecessary.

Optimized Code

$t1 = m * n$

$t2 = p + q$

$t4 = t2 - t1$

This is called **Common Subexpression Elimination**.

Benefits

Optimization:

- Removes redundant calculations.
- Reduces the number of instructions.
- Reduces execution time.
- Improves processor efficiency.
- Can reduce energy consumption.

Conclusion: Reusing $t1$ instead of calculating $m * n$ again makes the intermediate code more efficient.

Q4. Control Flow and Boolean Expression Handling

Given the Boolean expression: $\text{if } (x > y \parallel p < q)$

Intermediate code: $\text{if } x > y \text{ goto L1 if } p < q \text{ goto L1 goto}$

L2

L1: ...

Interpret the control flow of the given intermediate representation and explain how shortcircuit evaluation minimizes unnecessary condition checking during execution.

ANSWER:

Control Flow and Boolean Expression Handling

Given:

if ($x > y \parallel p < q$)

Intermediate code:

if $x > y$ goto L1 if

$p < q$ goto L1 goto

L2

L1: ...

Control Flow

The compiler first evaluates:

$x > y$

Case 1: $x > y$ is TRUE

Control immediately jumps to:

L1

There is no need to evaluate $p < q$.

Case 2: $x > y$ is FALSE

The compiler then evaluates:

$p < q$

If this is true, control jumps to L1.

Case 3: Both Conditions are FALSE

If:

$x > y = \text{false}$ p

$< q = \text{false}$

control goes to:

L2

Short-Circuit Evaluation

This is called **short-circuit evaluation**.

For the OR operator ($\parallel$), if the first condition is already true, the entire expression is true. Therefore, the second condition does not need to be checked.

Benefit

Short-circuit evaluation:

- Avoids unnecessary condition checking.
- Reduces the number of operations.
- Improves execution efficiency.
- Can be especially useful when the second condition is expensive to evaluate.

Conclusion: The given intermediate code correctly implements short-circuit evaluation for the OR operator.

Q5. Backpatching and Flow Control Given:

Instruction	Target
-------------	--------

200: if $x == y$ goto _	
-------------------------	--

201: goto _	
-------------	--

Backpatch lists:

- truelist = {200}
- falselist = {201}

Analyze the role of backpatching in intermediate code generation. Demonstrate how target addresses are updated during control flow handling and explain its importance in Boolean expression translation.

ANSWER:

Backpatching and Flow Control

Given:

200: if x==y goto _

201: goto _

Backpatch lists:

truelist = {200}

falselist = {201}

Role of Backpatching

Backpatching is a technique used in intermediate code generation to fill in incomplete jump targets.

When the compiler initially generates the code, the final target addresses may not yet be known. Therefore, it temporarily leaves the target as _.

The compiler maintains:

- truelist → instructions that should jump when the condition is **true**.
- falselist → instructions that should jump when the condition is **false**.

Here:

truelist = {200}

means instruction 200 needs the address of the true branch.

falselist = {201}

means instruction 201 needs the address of the false branch.

Example of Backpatching

Suppose the true branch starts at address 210 and the false branch starts at address 220.

Before backpatching:

200: if x==y goto _

201: goto _

After backpatching:

200: if x==y goto 210

201: goto 220

Therefore:

truelist {200} → 210

falselist {201} → 220

Importance in Boolean Expression Translation

Backpatching is important because:

- It allows the compiler to generate jumps before their targets are known.
- It supports conditional statements.
- It supports Boolean expressions.

- It helps construct control-flow graphs.
- It resolves forward jumps efficiently.

Conclusion: Backpatching connects incomplete intermediate-code jumps with their final target addresses and is an important technique for Boolean expression and control-flow translation.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4	Level 3	Level 2	Level 1
		(Exemplary)	(Proficient)	(Developing)	(Beginning)
Understanding of Data	25%	Accurately interprets all data patterns, trends, and relationships	Interprets data correctly with minor gaps	Partial understanding; misses key trends	Misinterprets or fails to understand data
Analysis	25%	Provides deep analysis with strong logical reasoning and justification	Provides reasonable analysis with some justification	Basic analysis with weak reasoning	No meaningful analysis provided
Application of Concepts	20%	Effectively applies relevant concepts/tools to interpret data accurately	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply concepts to data
Conclusion	20%	Draws clear, meaningful conclusions with strong insights and implications	Provides valid conclusions with moderate insights	Conclusions are vague or weak	No clear or valid conclusions
Clarity	10%	Presents interpretation clearly using proper structure, visuals, and terminology	Generally clear with minor issues	Lacks clarity or proper structure	Poor presentation and difficult to understand